COMP3308

Xiyuan Pan

May 2026

1 Lecture Note / Algorithm Knowledge Base

1.1 Rational Agents and AI Basics

Core Idea

The course adopts the rational-agent view of AI: an intelligent agent perceives its environment, uses its knowledge and percept history, and chooses actions that maximize a performance measure. In complex settings, perfect rationality may be infeasible, so agents may act with limited rationality.

Input and Output

- Input: percept sequence, built-in knowledge, goals, and a performance measure.
- Output: an action intended to maximize expected performance.

Step-by-step Procedure

1. Observe the current percepts from the environment.
2. Combine them with previous percepts, goals, and available knowledge.
3. Evaluate possible actions using the performance measure.
4. Select the action judged best under the available computation.

Formula and Calculation Details

No explicit formula is introduced in the lecture for rational agents. The key concept is:

acting rationally = choosing the action that maximizes the performance measure.

Worked Mini Example

Suppose a vacuum agent can either `clean` or `move`. If the current square is dirty and the performance measure rewards clean squares, then:

1. Percept: current square is dirty.
2. Candidate actions: `clean`, `move`.
3. `clean` immediately improves the score.
4. The rational action is `clean`.

Strengths and Weaknesses

- Strengths: general framework; separates goals, knowledge, and action choice.
- Weaknesses: defining a good performance measure can be difficult; exact best action may be too expensive to compute.

Exam Talking Points

- "The course uses the rational-agent approach: AI systems act to maximize a performance measure."
- "Limited rationality is needed when exact optimal reasoning is computationally infeasible."

Common Mistakes

- Equating rational behavior with human-like behavior.
- Assuming rational agents always compute the globally optimal action in practice.

1.2 Breadth-First Search (BFS)

Core Idea

BFS expands the shallowest unexpanded node first. It explores the search tree level by level, so it finds the shallowest goal before any deeper one.

Input and Output

- Input: initial state, successor function, goal test, and a search tree or graph.
- Output: a path to a goal state, if one is found.

Step-by-step Procedure

1. Put the initial node in the fringe.
2. Remove the first node from the fringe.
3. If it is a goal, return its path.
4. Otherwise expand it and append its children to the end of the fringe.
5. Repeat until a goal is found or the fringe is empty.

Formula and Calculation Details

$$1 + b + b^2 + \cdots + b^d = O(b^d)$$

where b is the branching factor and d is the depth of the shallowest goal. This is both the time and space complexity of BFS in the lecture setting.

Worked Mini Example

Tree: A has children B, C; B has D, E; C has F, G; G is a goal.

1. Fringe = $[A]$.
2. Expand A: fringe = $[B, C]$.
3. Expand B: fringe = $[C, D, E]$.
4. Expand C: fringe = $[D, E, F, G]$.
5. Expand D, E, F in order, then encounter G.
6. Expansion order: A, B, C, D, E, F, G; solution path: $A \to C \to G$.

Strengths and Weaknesses

- Strengths: complete when b is finite; optimal when all step costs are equal.
- Weaknesses: exponential time and memory; not optimal when step costs differ.

Exam Talking Points

- "BFS expands the shallowest node first and is optimal only when all step costs are equal."
- "Its main practical limitation is memory: $O(b^d)$ nodes must be stored."

Common Mistakes

- Claiming BFS is always optimal.
- Forgetting that children are appended to the end of the fringe.

1.3 Uniform-Cost Search (UCS)

Core Idea

UCS expands the node with the smallest path cost so far. Unlike BFS, it cares about accumulated cost, not depth.

Input and Output

- Input: initial state, successor function, positive step costs, and goal test.
- Output: a least-cost path to a goal.

Step-by-step Procedure

1. Put the start node in a priority queue ordered by path cost $g(n)$.
2. Remove the node with the smallest $g(n)$.
3. If it is a goal, return its path.
4. Otherwise expand it and insert children with their updated path costs.
5. Repeat.

Formula and Calculation Details

$$g(n) = \text{cost from the root to node } n.$$

$$O\left(b^{1+\lfloor C^*/\varepsilon \rfloor}\right)$$

is the lecture's complexity characterization, where C^* is the optimal solution cost and ε is the smallest positive step cost.

Worked Mini Example

Let $S \to A$ cost 2, $S \to B$ cost 1, $A \to G$ cost 2, $B \to G$ cost 10.

1. Start: fringe $= [S : 0]$.
2. Expand S: fringe $= [B : 1, A : 2]$.
3. Expand B: fringe $= [A : 2, G : 11]$.
4. Expand A: fringe $= [G : 4, G : 11]$.
5. Remove $G : 4$ first; solution is $S \to A \to G$ with total cost 4.

Strengths and Weaknesses

- Strengths: complete for positive step costs; optimal.
- Weaknesses: can explore many cheap partial paths before useful expensive ones; large memory demand.

Exam Talking Points

- "UCS is BFS generalized to unequal step costs."
- "It always expands the node with the smallest accumulated path cost $g(n)$."

Common Mistakes

- Ordering by the most recent edge cost instead of total path cost.
- Forgetting that UCS requires positive step costs for completeness.

1.4 Depth-First, Depth-Limited, and Iterative Deepening Search

Core Idea

DFS expands the deepest unexpanded node first. Depth-limited search adds a cutoff depth. Iterative deepening search (IDS) repeatedly runs depth-limited DFS with increasing limits, combining BFS-like completeness with DFS-like low memory.

Input and Output

- Input: initial state, successor function, goal test; IDS also uses increasing depth limits.
- Output: a path to a goal, if one is reached.

Step-by-step Procedure

1. DFS: remove the first node from the fringe, expand it, and place successors at the front.
2. Depth-limited search: do DFS but stop expanding below limit ℓ.
3. IDS: run depth-limited search with $\ell = 0, 1, 2, \ldots$ until a goal is found.

Formula and Calculation Details

$$\text{DFS time} = 1 + b + \cdots + b^m = O(b^m), \qquad \text{DFS space} = O(bm),$$

where m is the maximum search depth.

$$\text{Depth-limited time} = O(b^\ell), \qquad \text{space} = O(b\ell).$$

$$(d+1)b^0 + db^1 + (d-1)b^2 + \cdots + b^d = O(b^d)$$

for IDS, where d is the shallowest solution depth.

Worked Mini Example

Use the tree $A \to \{B, C\}$, $B \to \{D, E\}$, $C \to \{F, G\}$, with G as goal.

1. DFS expansion order with left children first: A, B, D, E, C, F, G.
2. Depth-limited search with $\ell = 1$ visits only A, B, C.
3. IDS tries $\ell = 0$: A; then $\ell = 1$: A, B, C; then $\ell = 2$: finds G.

Strengths and Weaknesses

- Strengths: DFS uses little memory; IDS is complete and, for unit costs, optimal.
- Weaknesses: DFS is not optimal and can fail in infinite-depth spaces; IDS repeats shallow expansions.

Exam Talking Points

- "IDS is usually preferred when the solution depth is unknown because it keeps DFS memory but gains BFS-style completeness."
- "DFS is attractive for memory, not for optimality."

Common Mistakes

- Saying DFS is complete without stating the finite-depth condition.
- Forgetting that IDS overhead is usually modest because most nodes are near the leaves.

1.5 Greedy Best-First Search

Core Idea

Greedy best-first search uses only a heuristic estimate of distance to the goal and expands the node that appears closest.

Input and Output

- Input: initial state, successor function, goal test, and heuristic $h(n)$.
- Output: a path to a goal, not necessarily the cheapest one.

Step-by-step Procedure

1. Insert the start node into a priority queue ordered by $h(n)$.
2. Remove the node with smallest $h(n)$.
3. If it is a goal, return its path.
4. Otherwise expand it and insert successors ordered by their h values.

Formula and Calculation Details

$$h(n) = \text{estimated cost from node } n \text{ to a goal.}$$

Greedy search minimizes $h(n)$ only. In the lecture example, $h(n)$ is straight-line distance to Bucharest.

Worked Mini Example

Suppose S has children A and B with $h(A) = 2$, $h(B) = 5$. A leads to a goal with path cost 10; B leads to a goal with path cost 4.

1. Greedy search chooses A first because $2 < 5$.
2. It reaches the goal through A with cost 10.
3. Although a cheaper path through B exists, it is ignored because greedy search used only h.

Strengths and Weaknesses

- Strengths: can be much faster than uninformed search when h is informative.
- Weaknesses: not optimal; incomplete in infinite spaces; can loop or chase misleading heuristics.

Exam Talking Points

- "Greedy search minimizes estimated remaining cost, not total solution cost."
- "A good heuristic may help dramatically, but optimality is not guaranteed."

Common Mistakes

- Confusing greedy search with UCS or A*.
- Using $g(n)$ and calling the result greedy search; that becomes UCS behavior.

1.6 A* Search

Core Idea

A* balances what has already been spent and what is estimated to remain. It expands the node with the smallest estimated total path cost.

Input and Output

- Input: initial state, successor function, goal test, step costs, heuristic $h(n)$.
- Output: an optimal path when the heuristic satisfies the required condition.

Step-by-step Procedure

1. Put the start node in a priority queue ordered by $f(n)$.
2. Remove the node with smallest $f(n)$.
3. If it is a goal, return the path.
4. Otherwise expand it and insert successors with updated g, h, and f.
5. Continue until a goal is selected for expansion.

Formula and Calculation Details

$$f(n) = g(n) + h(n),$$

where $g(n)$ is cost so far and $h(n)$ is estimated cost to a goal.

$$h(n) \leq h^*(n)$$

defines an admissible heuristic, where $h^*(n)$ is the true remaining cost.

$$h(n_i) \leq c(n_i, n_j) + h(n_j)$$

defines consistency for parent n_i, child n_j, and step cost $c(n_i, n_j)$.

$$h(n) = \max\{h_1(n), \ldots, h_m(n)\}$$

is an admissible composite heuristic if all h_i are admissible.

Worked Mini Example

Let $S \to A$ cost 2, $S \to B$ cost 1, $A \to G$ cost 4, $B \to G$ cost 10, with $h(A) = 3$, $h(B) = 1$, and $h(G) = 0$.

1. For A: $g = 2$, $h = 3$, $f = 5$.
2. For B: $g = 1$, $h = 1$, $f = 2$.
3. A* expands B first, then discovers G with $g = 11$, $f = 11$.
4. It still keeps A with $f = 5$, expands A, then reaches G with $g = 6$.
5. The optimal path returned is $S \to A \to G$ with cost 6.

Strengths and Weaknesses

- Strengths: optimal with admissible heuristics; optimally efficient with consistent heuristics.
- Weaknesses: exponential memory/time in hard problems; strong heuristics may themselves be expensive to compute.

Exam Talking Points

- "A* combines UCS and greedy search using $f(n) = g(n) + h(n)$."
- "Admissibility supports optimality; consistency makes f non-decreasing along paths."

Common Mistakes

- Returning a goal as soon as it is generated rather than when selected for expansion.
- Assuming admissible automatically means consistent.

1.7 Hill-Climbing

Core Idea

Hill-climbing keeps one current state and repeatedly moves to the best neighboring state. It is an iterative improvement method for optimization rather than systematic path finding.

Input and Output

- Input: initial state, neighbor generator, evaluation function v.
- Output: a local optimum, which may or may not be global.

Step-by-step Procedure

1. Set the current node to the initial state.
2. Generate all neighbors.
3. Choose the best neighbor according to v.
4. If it is not better than the current node, stop.
5. Otherwise move to that neighbor and repeat.

Formula and Calculation Details

For minimization, accept a child only when:

$$v(\text{child}) < v(\text{current}).$$

The lecture uses v as the score being minimized or maximized depending on the version.

Worked Mini Example

Scores to minimize: $S(7)$ has neighbors $A(4)$, $B(5)$, $C(8)$; A has $D(3)$, $E(7)$; D has no better child.

1. Start at $S(7)$.
2. Best neighbor is $A(4)$, so move to A.
3. Best neighbor is $D(3)$, so move to D.
4. No neighbor improves on 3, so stop at D.

Strengths and Weaknesses

- Strengths: tiny memory footprint; often useful in huge spaces.
- Weaknesses: not complete, not optimal, can get stuck on local optima, plateaus, and ridges.

Exam Talking Points

- "Hill-climbing trades guarantees for speed and memory efficiency."
- "Random restarts help when the quality of the local optimum depends on the starting point."

Common Mistakes

- Describing it as a graph-search method with backtracking.
- Forgetting whether the problem is ascent or descent.

1.8 Beam Search

Core Idea

Beam search keeps only the best k states at each level instead of all states, making search more memory-efficient but less reliable.

Input and Output

- Input: one or more initial states, beam width k, successor function, evaluation function.
- Output: a goal if one survives pruning.

Step-by-step Procedure

1. Start from one state or k random states.
2. Generate all successors of the current retained states.
3. If any successor is a goal, stop.
4. Otherwise keep only the best k successors.
5. Repeat.

Formula and Calculation Details

No special formula is used beyond the beam width k, the number of retained states per level.

Worked Mini Example

With $k = 2$, from $S(7)$ generate $A(4), B(5), C(8)$.

1. Keep A and B.
2. Generate their children $D(3), E(7), F(2), H(1)$.
3. Keep H and F because they are the best two.

Strengths and Weaknesses

- Strengths: much lower memory than full best-first search; can explore multiple regions at once.
- Weaknesses: not complete and not optimal; promising states may be discarded too early.

Exam Talking Points

- "Beam search is a width-limited search: it keeps only the best k states."
- "Using beam search with A* saves memory but sacrifices completeness and optimality."

Common Mistakes

- Confusing beam width k with branching factor b.
- Assuming the discarded states can later be recovered.

1.9 Simulated Annealing

Core Idea

Simulated annealing sometimes accepts worse moves so it can escape poor local optima. Early in the run, bad moves are allowed more often; later, the algorithm behaves more like hill-climbing.

Input and Output

- Input: initial state, neighbor generator, evaluation function v, cooling schedule for temperature T.
- Output: a state of good quality, potentially the global optimum under very slow cooling.

Step-by-step Procedure

1. Start from an initial state.
2. Randomly choose one successor.
3. If it is better, accept it.
4. If it is worse, accept it with probability p.
5. Lower the temperature according to the schedule and repeat.

Formula and Calculation Details

For minimization, the lecture uses:

$$p = e^{\frac{v(n)-v(m)}{T}},$$

where n is the current state, m is the candidate child, $v(\cdot)$ is the score, and T is temperature. If m is worse, then $v(m) > v(n)$ and $p < 1$.

$$T \leftarrow 0.8T$$

is an example cooling schedule shown in the lecture.

Worked Mini Example

Current score $v(n) = 10$, candidate score $v(m) = 11$, temperature $T = 2$.

1. The move is worse because $11 > 10$.
2. Compute $p = e^{(10-11)/2} = e^{-0.5} \approx 0.607$.
3. Draw $p' = 0.40$ uniformly from $[0, 1]$.
4. Since $0.607 > 0.40$, accept the worse move.

Strengths and Weaknesses

- Strengths: can escape local minima; easy to implement.
- Weaknesses: performance depends strongly on the cooling schedule; theoretically good schedules may be impractically slow.

Exam Talking Points

- "The temperature controls exploration: high T permits many bad moves, low T makes the search greedy."
- "With a sufficiently slow schedule, simulated annealing can reach the global optimum."

Common Mistakes

- Using the wrong sign in the acceptance probability.
- Saying all worse moves are rejected.

1.10 Genetic Algorithms

Core Idea

Genetic algorithms maintain a population of candidate solutions and evolve it through selection, crossover, and mutation.

Input and Output

- Input: encoded population, fitness function, crossover rule, mutation rate, stopping rule.
- Output: the fittest individual found.

Step-by-step Procedure

1. Initialize a population of individuals.
2. Compute each individual's fitness.
3. Select parents with probability proportional to fitness.
4. Recombine parents using crossover to create children.
5. Mutate children with small probability.
6. Replace the old population and repeat.

Formula and Calculation Details

$$P_i = \frac{f(S_i)}{\sum_j f(S_j)},$$

where P_i is the reproduction probability of individual S_i and $f(S_i)$ is its fitness.

Worked Mini Example

Population fitnesses: $f(S_1) = 2$, $f(S_2) = 3$, $f(S_3) = 5$.

1. Sum of fitnesses: $2 + 3 + 5 = 10$.
2. Reproduction probabilities: $P_1 = 0.2$, $P_2 = 0.3$, $P_3 = 0.5$.
3. A random draw is most likely to choose S_3 as a parent.
4. Two selected parents crossover, then a bit may mutate with a small probability.

Strengths and Weaknesses

- Strengths: explores multiple candidate regions at once; crossover shares useful partial structure.
- Weaknesses: not complete or optimal; success depends heavily on representation and parameter choices.

Exam Talking Points

- "Genetic algorithms combine uphill pressure from fitness with random exploration from crossover and mutation."
- "They are closely related to stochastic beam search."

Common Mistakes

- Treating mutation as the main source of improvement instead of crossover plus selection.
- Forgetting that the encoding of solutions matters.

1.11 Minimax

Core Idea

Minimax chooses the best move for a player in a two-player, deterministic, perfect-information game, assuming the opponent also plays optimally.

Input and Output

- Input: current game state, successor function, terminal test, and utility function.
- Output: the move with the best minimax value for the current player.

Step-by-step Procedure

1. Build the game tree from the current state.
2. Evaluate terminal states with the utility function.
3. At MIN nodes, back up the minimum child value.
4. At MAX nodes, back up the maximum child value.
5. Choose the root action that gives MAX the highest backed-up value.

Formula and Calculation Details

$$\text{MINIMAX}(s) = \begin{cases} U(s), & \text{if } s \text{ is terminal,} \\ \max_{s' \in \text{Succ}(s)} \text{MINIMAX}(s'), & \text{if } s \text{ is a MAX node,} \\ \min_{s' \in \text{Succ}(s)} \text{MINIMAX}(s'), & \text{if } s \text{ is a MIN node,} \end{cases}$$

where $U(s)$ is terminal utility and $\text{Succ}(s)$ is the set of successor states.

$$\text{Time} = O(b^m), \qquad \text{Space} = O(bm),$$

with branching factor b and maximum tree depth m.

Worked Mini Example

MAX has two moves:

$$A \to \{3, 5\}, \qquad B \to \{2, 9\}.$$

1. MIN evaluates A as $\min(3, 5) = 3$.
2. MIN evaluates B as $\min(2, 9) = 2$.
3. MAX chooses $\max(3, 2) = 3$.
4. Therefore MAX selects move A.

Strengths and Weaknesses

- Strengths: gives optimal play under its assumptions.
- Weaknesses: exponential cost; unrealistic to search entire trees in large games.

Exam Talking Points

- "Minimax maximizes MAX's worst-case outcome under optimal opponent play."
- "It is normally implemented as depth-first search with values propagated upward."

Common Mistakes

- Taking a maximum at MIN nodes.
- Forgetting that the guarantee assumes an optimal opponent.

1.12 Alpha-Beta Pruning

Core Idea

Alpha-beta pruning returns the same move as minimax while skipping branches that cannot affect the final choice.

Input and Output

- Input: same game tree inputs as minimax.
- Output: the same best move and root value as minimax, with fewer evaluations.

Step-by-step Procedure

1. Search the tree depth-first.
2. Track α, the best value already found for MAX on the path.
3. Track β, the best value already found for MIN on the path.
4. At MAX nodes, update α; at MIN nodes, update β.
5. Prune when a MAX node reaches $\alpha \geq \beta$ or a MIN node reaches $\beta \leq \alpha$.

Formula and Calculation Details

α = best lower bound for MAX so far, β = best upper bound for MIN so far.

Best-case node evaluations with perfect ordering:

$$O(b^{d/2}),$$

compared with $O(b^d)$ for plain minimax.

Worked Mini Example

Suppose root MAX has left child A and right child B.

$$A = \min(200, 100) = 100.$$

At B, the first child gives value 20.

1. After exploring A, root $\alpha = 100$.
2. At B (a MIN node), $\beta = 20$ after the first child.
3. Since $\beta = 20 \leq \alpha = 100$, the remaining children of B cannot improve the root decision.
4. Prune them; the root still chooses A with value 100.

Strengths and Weaknesses

- Strengths: never changes the minimax answer; good move ordering can roughly double practical look-ahead depth.
- Weaknesses: worst-case behavior is still minimax when ordering is poor.

Exam Talking Points

- "Alpha-beta pruning removes branches that are provably irrelevant to the final minimax choice."
- "Move ordering matters: perfect ordering gives the largest pruning benefit."

Common Mistakes

- Thinking alpha-beta changes the chosen move.
- Applying pruning without keeping track of ancestor bounds.

1.13 Expectiminimax

Core Idea

Expectiminimax extends minimax to non-deterministic games by adding chance nodes whose values are expected utilities.

Input and Output

- Input: game tree with MAX, MIN, and chance nodes; transition probabilities; utility function.
- Output: the best move under expected utility.

Step-by-step Procedure

1. Evaluate terminal states with utility.
2. At MAX nodes, take the maximum child value.
3. At MIN nodes, take the minimum child value.
4. At chance nodes, compute the probability-weighted average of child values.
5. Choose the root move with the best expectiminimax value.

Formula and Calculation Details

$$V(\text{chance node}) = \sum_i p_i V_i,$$

where p_i is the probability of outcome i and V_i is the child value.

$$\text{Time} = O(b^m n^m),$$

where n is the number of distinct random outcomes per chance event in the lecture notation.

Worked Mini Example

A chance node has two children with values 2 and 4, each with probability 0.5.

$$V = 0.5(2) + 0.5(4) = 3.$$

If MAX compares this move to another move worth 2.5, MAX chooses the expected-value-3 move.

Strengths and Weaknesses

- Strengths: handles games with random outcomes such as dice rolls.
- Weaknesses: more expensive than minimax; expected utility depends on known probabilities.

Exam Talking Points

- "Chance nodes are evaluated by weighted averages, not by min or max."
- "Expectiminimax is the appropriate extension of minimax for non-deterministic games."

Common Mistakes

- Taking an unweighted average when outcomes are not equally likely.
- Forgetting that MIN and MAX nodes still behave as in minimax.

1.14 Machine Learning Basics

Core Idea

Machine learning builds models from data so that they can make predictions on unseen examples. The lecture distinguishes supervised learning, unsupervised learning, and reinforcement learning; supervised tasks include classification and regression.

Input and Output

- Supervised input: labelled examples (x, y).
- Supervised output: a learned function mapping $x \mapsto y$.
- Unsupervised input: unlabelled examples x.
- Unsupervised output: structure such as clusters.

Step-by-step Procedure

1. Collect examples.
2. Decide the task type: classification for discrete labels, regression for numeric targets, clustering for unlabelled grouping.
3. Choose a learning algorithm.
4. Train the model on training data.
5. Evaluate on unseen data.

Formula and Calculation Details

The lecture gives a regression example:

$$\text{PRP} = -56.1 + 0.049\,\text{MYCT} + 0.015\,\text{MMIN} + 0.006\,\text{MMAX} + 0.630\,\text{CACH} - 0.270\,\text{CHMIN} + 1.46\,\text{CHMAX},$$

where PRP is predicted computer performance and the other symbols are input attributes. This illustrates a learned numeric mapping for regression.

Worked Mini Example

Given house examples with features $x = (\text{size}, \text{bedrooms})$ and price y:

1. If y is a price, the task is regression.
2. If y is instead `cheap`/`expensive`, the task is classification.
3. If no y values are given and we group similar houses, the task is unsupervised clustering.

Strengths and Weaknesses

- Strengths: learns from data rather than hand-coding every rule.
- Weaknesses: quality depends on data quality, task definition, and evaluation design.

Exam Talking Points

- "Classification predicts a discrete class; regression predicts a numeric value."
- "Supervised learning uses labelled examples; unsupervised learning has no target outputs."

Common Mistakes

- Calling every prediction task classification.
- Saying clustering is supervised.

1.15 Clustering (briefly covered)

Core Idea

Clustering groups unlabelled examples so that items in the same cluster are similar and items in different clusters are dissimilar.

Input and Output

- Input: unlabelled vectors x.
- Output: a finite set of clusters.

Step-by-step Procedure

1. Represent each example as a feature vector.
2. Measure similarity or distance between examples.
3. Group nearby or similar examples together.
4. Return the discovered groups.

Formula and Calculation Details

No clustering formula is given in the lecture material. The lecture only names examples such as k-means, nearest-neighbor clustering, and hierarchical clustering.

Worked Mini Example

Points $(1,1)$, $(1,2)$, $(8,8)$, $(9,8)$ naturally form two groups:

1. $(1,1)$ and $(1,2)$ are close together.
2. $(8,8)$ and $(9,8)$ are close together.
3. A reasonable clustering is $\{(1,1),(1,2)\}$ and $\{(8,8),(9,8)\}$.

Strengths and Weaknesses

- Strengths: useful when labels are unavailable.
- Weaknesses: "good" clusters may be ambiguous and depend on the chosen similarity measure.

Exam Talking Points

- "Clustering is unsupervised because the data has no target labels."
- "Its goal is high within-cluster similarity and low between-cluster similarity."

Common Mistakes

- Treating cluster IDs as pre-given labels.
- Assuming the lecture covered a specific clustering algorithm in detail.

1.16 Classifier Evaluation, Confusion Matrix, and Cross-Validation

Core Idea

Classifier quality must be estimated on unseen data. The lectures cover holdout, stratification, repeated holdout, cross-validation, confusion matrices, accuracy, precision, recall, F_1, significance testing, and cost-sensitive evaluation.

Input and Output

- Input: labelled data and one or more classifiers.
- Output: performance estimates and, when needed, a statistical comparison.

Step-by-step Procedure

1. Split data into training and test sets, optionally adding a validation set.
2. Use stratification so class proportions remain similar across splits.
3. Train only on training data.
4. Evaluate on unseen test folds or test sets.
5. Build a confusion matrix and compute relevant metrics.
6. For model comparison, use paired fold-wise differences and a paired t procedure.

Formula and Calculation Details

$$\text{accuracy} = \frac{\text{correct predictions}}{\text{all predictions}}, \qquad \text{error rate} = 1 - \text{accuracy}.$$

For a binary confusion matrix:

$$\text{accuracy} = \frac{tp + tn}{tp + tn + fp + fn},$$

$$P = \frac{tp}{tp + fp}, \qquad R = \frac{tp}{tp + fn}, \qquad F_1 = \frac{2PR}{P + R}.$$

For S-fold cross-validation:

$$\text{CV accuracy} = \frac{1}{S} \sum_{i=1}^{S} \text{acc}_i.$$

For paired classifier comparison:

$$\sigma = \sqrt{\frac{\sum_i (d_i - d_{\text{mean}})^2}{k - 1}},$$

$$Z = d_{\text{mean}} \pm t_{1-\alpha,k-1} \frac{\sigma}{\sqrt{k}},$$

where d_i is the fold-wise difference, k is the number of folds, and $t_{1-\alpha,k-1}$ is the critical t value.

Worked Mini Example

Suppose:

$$tp = 24, \quad fn = 1, \quad fp = 70, \quad tn = 5.$$

1. Accuracy $= (24 + 5)/(24 + 1 + 70 + 5) = 29/100 = 0.29$.
2. Precision $= 24/(24 + 70) = 24/94 \approx 0.255$.
3. Recall $= 24/(24 + 1) = 24/25 = 0.96$.
4. $F_1 = 2(0.255)(0.96)/(0.255 + 0.96) \approx 0.403$.

Strengths and Weaknesses

- Strengths: cross-validation uses data efficiently; precision/recall reveal class-specific behavior hidden by accuracy.
- Weaknesses: repeated training is computationally expensive; metrics can conflict; raw accuracy can be misleading under class imbalance or unequal costs.

Exam Talking Points

- "Test data must not be used to train or tune the classifier."
- "Precision answers 'when the classifier predicts positive, how often is it right?'; recall answers 'of all real positives, how many were found?"'
- "Cross-validation avoids overlapping test sets and gives a more stable estimate than one holdout split."

Common Mistakes

- Tuning on the test set.
- Mixing up precision and recall denominators.
- Forgetting to stratify when classes are imbalanced.

1.17 k-Nearest Neighbours (kNN)

Core Idea

kNN stores the training data and predicts a new example from the labels of nearby examples. It is a lazy, instance-based learner.

Input and Output

- Input: labelled training examples, a new example, a distance function, and a chosen k.
- Output: a predicted class by majority vote or a numeric prediction by averaging.

Step-by-step Procedure

1. Normalize numeric attributes when scales differ.
2. Compute the distance from the new example to every training example.
3. Select the k nearest examples.
4. For classification, return the majority class.
5. For regression, average the target values or use distance-weighted averaging.

Formula and Calculation Details

$$D_{\text{Euclidean}}(A,B) = \sqrt{(a_1-b_1)^2 + \cdots + (a_n-b_n)^2},$$
$$D_{\text{Manhattan}}(A,B) = |a_1-b_1| + \cdots + |a_n-b_n|,$$
$$D_{\text{Minkowski}}(A,B) = \left(|a_1-b_1|^q + \cdots + |a_n-b_n|^q\right)^{1/q}.$$

For normalization:

$$a_i = \frac{v_i - \min v_i}{\max v_i - \min v_i},$$

where v_i is the raw value of attribute i. For regression:

$$\hat{y} = \frac{y_1 + \cdots + y_k}{k}.$$

For weighted nearest neighbour:

$$w_i = \frac{1}{D(X_{\text{new}}, X_i)}, \qquad \hat{y} = \sum_i w_i y_i$$

as presented in the lecture.

Worked Mini Example

Training values for one feature:

$$(4, \text{low}),\ (9, \text{medium}),\ (8, \text{medium}),\ (15, \text{high}),\ (7, \text{medium}).$$

Predict for $x = 5$ using Manhattan distance.

1. Distances: $1, 4, 3, 10, 2$.
2. For 1NN, nearest is 4 with label `low`; prediction is `low`.
3. For 3NN, nearest labels are `low`, `medium`, `medium`; prediction is `medium`.

Strengths and Weaknesses

- Strengths: simple, no training cost beyond storage, robust with suitable k.
- Weaknesses: slow prediction, sensitive to irrelevant features and scale, affected by the curse of dimensionality.

Exam Talking Points

- "kNN is lazy learning: the model is effectively built at query time."
- "Choosing larger k can reduce sensitivity to noisy single examples."

Common Mistakes

- Forgetting to normalize numeric features.
- Using k larger than is sensible for the training size.
- Ignoring how missing values are handled in distance calculations.

1.18 1R

Core Idea

1R builds one simple rule using only one attribute: for each attribute value, predict the most frequent class seen in training data, then choose the attribute with the smallest training error.

Input and Output

- Input: labelled training data.
- Output: a single-attribute rule set.

Step-by-step Procedure

1. For each attribute, inspect each of its values.
2. For each value, count class frequencies.
3. Assign the most frequent class to that value.
4. Count the rule's training errors.
5. Choose the rule with the fewest errors.

Formula and Calculation Details

The lecture uses misclassification count or error rate as the preference function:

$$\text{error rate} = \frac{\text{number of misclassified examples}}{\text{number of examples}}.$$

For numeric attributes, adjacent class changes create candidate breakpoints halfway between values:

$$\text{breakpoint} = \frac{v_i + v_{i+1}}{2}.$$

Worked Mini Example

For attribute `humidity`:

$$\texttt{high} : 3 \text{ yes}, 4 \text{ no}; \qquad \texttt{normal} : 6 \text{ yes}, 1 \text{ no}.$$

1. Predict `no` for `high`.
2. Predict `yes` for `normal`.
3. Errors are $3 + 1 = 4$.
4. Repeat for all attributes and choose the attribute with smallest total error.

Strengths and Weaknesses

- Strengths: extremely simple, interpretable, and computationally cheap.
- Weaknesses: can underfit because it uses only one attribute; numeric discretization can overfit if too many intervals are created.

Exam Talking Points

- "1R is a useful simple baseline because it searches all one-attribute rules."
- "It chooses the attribute whose rule has the lowest training error."

Common Mistakes

- Choosing the attribute with the most values instead of the lowest error.
- Forgetting that numeric data must be discretized first.

1.19 Naive Bayes

Core Idea

Naive Bayes classifies by comparing posterior probabilities for each class while assuming that attributes are conditionally independent given the class.

Input and Output

- Input: labelled training data and a new example with feature values.
- Output: the class with highest posterior probability.

Step-by-step Procedure

1. Estimate class prior probabilities from training data.
2. Estimate conditional probabilities for each attribute value given each class.
3. Multiply the conditional probabilities together for a new example.
4. Multiply by the class prior.
5. Choose the class with the largest resulting posterior score.

Formula and Calculation Details

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E)}.$$

Under the naive independence assumption:

$$P(E \mid H) = \prod_{j=1}^{m} P(E_j \mid H).$$

For classification, the denominator is common to all classes, so:

$$\hat{c} = \arg\max_{c} P(c) \prod_{j} P(x_j \mid c).$$

Laplace correction:

$$P(x = v \mid c) = \frac{\text{count}(x = v, c) + 1}{\text{count}(c) + k},$$

where k is the number of possible values of attribute x. The m-estimate generalization is:

$$P(x = v_i \mid c) = \frac{n_i + mp_i}{n + m}.$$

For numeric attributes with Gaussian assumption:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right),$$

where μ is the mean and σ is the standard deviation.

Worked Mini Example

Suppose:

$$P(\text{yes}) = \frac{9}{14}, \quad P(\text{sunny} \mid \text{yes}) = \frac{2}{9}, \quad P(\text{cool} \mid \text{yes}) = \frac{3}{9},$$

$$P(\text{no}) = \frac{5}{14}, \quad P(\text{sunny} \mid \text{no}) = \frac{3}{5}, \quad P(\text{cool} \mid \text{no}) = \frac{1}{5}.$$

For an example with `sunny` and `cool`:

1. Score for yes: $(9/14)(2/9)(3/9) = 1/21 \approx 0.0476$.
2. Score for no: $(5/14)(3/5)(1/5) = 3/70 \approx 0.0429$.
3. Since $0.0476 > 0.0429$, predict `yes`.

Strengths and Weaknesses

- Strengths: simple, fast, uses all attributes, often works surprisingly well.
- Weaknesses: correlated attributes violate assumptions; zero probabilities require smoothing; Gaussian assumption may be poor for numeric features.

Exam Talking Points

- "Naive Bayes assumes conditional independence of attributes given the class."
- "Laplace correction prevents a single zero count from forcing an entire class score to zero."

Common Mistakes

- Multiplying unconditional probabilities instead of class-conditional probabilities.
- Forgetting to omit missing attributes from the product when classifying.

1.20 Decision Trees

Core Idea

Decision trees recursively split the data using attributes that best reduce class impurity, producing a tree of tests whose leaves predict classes.

Input and Output

- Input: labelled training data.
- Output: a classification tree or equivalent set of rules.

Step-by-step Procedure

1. Choose the best attribute for the current node.
2. Split the data by attribute value.
3. Recurse on each subset.
4. Stop when a subset is pure, cannot be split further, or is empty.
5. Optionally prune to reduce overfitting.

Formula and Calculation Details

Entropy:

$$H(S) = -\sum_i P_i \log_2 P_i,$$

where P_i is the proportion of examples in class i. Conditional entropy after splitting on A:

$$H(S \mid A) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v).$$

Information gain:

$$\text{Gain}(S \mid A) = H(S) - H(S \mid A).$$

Split information:

$$\text{SplitInformation}(S \mid A) = -\sum_i \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}.$$

Gain ratio:

$$\text{GainRatio}(S \mid A) = \frac{\text{Gain}(S \mid A)}{\text{SplitInformation}(S \mid A)}.$$

Lecture cost-sensitive variants:

$$\frac{\text{Gain}(S \mid A)^2}{\text{Cost}(A)}$$

and

$$\frac{2^{\text{Gain}(S|A)} - 1}{(\text{Cost}(A) + 1)^w},$$

where $w \in [0, 1]$ controls the importance of test cost.

Worked Mini Example

Weather root: 9 yes, 5 no.

1. Root entropy:
$$H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940.$$
2. After splitting on `outlook`, the weighted remaining entropy is 0.639.
3. Gain:
$$0.940 - 0.639 = 0.247.$$
4. If this is the largest gain among candidate attributes, choose `outlook` at the root.

Strengths and Weaknesses

- Strengths: interpretable, efficient, handles mixed attribute types with preprocessing.
- Weaknesses: prone to overfitting; information gain favors highly branching attributes; trees can become unstable.

Exam Talking Points

- "Entropy measures impurity; information gain measures reduction in impurity after a split."
- "Pruning improves generalization by removing branches that fit noise."

Common Mistakes

- Maximizing remaining entropy instead of minimizing it.
- Forgetting weighted averaging across branches.
- Ignoring validation data when discussing pruning.

1.21 Perceptron

Core Idea

A perceptron is a single-layer neural classifier with a linear decision boundary and a step activation function.

Input and Output

- Input: labelled vectors and current weights.
- Output: a binary class prediction and updated weights during training.

Step-by-step Procedure

1. Initialize weights and bias to small values.
2. Present one training example.
3. Compute the weighted sum and step output.
4. Compute the error $e = t - a$.
5. Update weights.
6. Repeat over epochs until all examples are correct or a limit is reached.

Formula and Calculation Details

$$n = wp + b = \sum_i w_i p_i + b,$$

$$a = \text{step}(n) = \begin{cases} 1, & n \geq 0, \\ 0, & n < 0, \end{cases}$$

$$e = t - a,$$

$$w_{\text{new}} = w_{\text{old}} + (t - a)p.$$

The decision boundary is:

$$wp + b = 0.$$

Worked Mini Example

Initial weights $w = [0, 0]$ and examples:

$$([1, 0], 1), \quad ([0, 1], 0), \quad ([1, 1], 1).$$

1. Example 1: $a = \text{step}(0) = 1$, correct, so $w = [0, 0]$.
2. Example 2: $a = \text{step}(0) = 1$, error $0 - 1 = -1$,

$$w = [0, 0] + (-1)[0, 1] = [0, -1].$$

3. Example 3: $a = \text{step}(-1) = 0$, error $1 - 0 = 1$,

$$w = [0, -1] + 1[1, 1] = [1, 0].$$

Strengths and Weaknesses

- Strengths: simple, efficient, converges for linearly separable data.
- Weaknesses: cannot solve non-linearly separable tasks such as XOR.

Exam Talking Points

- "A perceptron learns a linear decision boundary."
- "The convergence theorem applies only when the data is linearly separable."

Common Mistakes

- Forgetting the bias term.
- Claiming a perceptron can learn XOR.

1.22 Multilayer Neural Networks and Backpropagation

Core Idea

Multilayer perceptrons add hidden layers and differentiable activations so they can learn non-linear mappings. Backpropagation trains them by computing gradients from output layer back to earlier layers.

Input and Output

- Input: labelled examples, network architecture, initial weights, learning rate.
- Output: trained network weights.

Step-by-step Procedure

1. Choose the network architecture and initialize weights.
2. Forward propagate one example to compute outputs.
3. Compute output-layer errors.
4. Propagate errors backward through hidden layers.
5. Update weights using gradient descent.
6. Repeat until an error threshold, early stopping rule, or epoch limit is reached.

Formula and Calculation Details

Neuron activation:

$$a = f(wp + b).$$

Sigmoid:

$$f(x) = \frac{1}{1 + e^{-x}}, \qquad f'(x) = f(x)(1 - f(x)).$$

Sum of squared errors:

$$E = \frac{1}{2}\sum_i (d_i - o_i)^2.$$

Generic weight update:

$$w_{pq}(t+1) = w_{pq}(t) + \Delta w_{pq}(t),$$

$$\Delta w_{pq} = \eta\, \delta_q\, o_p,$$

where η is learning rate, o_p is the sending neuron's output, and δ_q is the error term for the receiving neuron. For output neurons:

$$\delta_q = (d_q - o_q) f'(net_q).$$

For hidden neurons:

$$\delta_q = f'(net_q) \sum_i w_{qi}\delta_i.$$

For sigmoid units specifically:

$$\delta_q = (d_q - o_q) o_q (1 - o_q)$$

at the output layer, and

$$\delta_q = o_q(1 - o_q) \sum_i w_{qi}\delta_i$$

at hidden layers. Momentum:

$$\Delta w_{pq}(t) = \eta\delta_q o_p + \mu\big(w_{pq}(t) - w_{pq}(t-1)\big),$$

where μ is the momentum coefficient.

Worked Mini Example

One output neuron has desired output $d = 1$, actual output $o = 0.8$, incoming activation $o_p = 0.5$, sigmoid unit, and $\eta = 0.1$.

1. Compute error term:

$$\delta = (1 - 0.8)(0.8)(1 - 0.8) = 0.032.$$

2. Weight change:

$$\Delta w = 0.1(0.032)(0.5) = 0.0016.$$

3. If old weight is 0.40, new weight is 0.4016.

Strengths and Weaknesses

- Strengths: can learn non-linear functions; very expressive.
- Weaknesses: training can be slow, sensitive to architecture and initialization, and prone to local minima and overfitting.

Exam Talking Points

- "Backpropagation applies gradient descent to the network error surface."
- "Hidden-layer error terms are computed from downstream error terms, hence the name backpropagation."

Common Mistakes

- Using a non-differentiable activation with backpropagation.
- Updating hidden units with target values that do not exist.
- Forgetting to separate training, validation, and test sets.

1.23 Deep Learning

Core Idea

Deep learning uses neural networks with multiple hidden layers to learn hierarchical feature representations, often directly from complex raw data.

Input and Output

- Input: large datasets, often high-dimensional signals such as images, audio, or text.
- Output: learned hierarchical features and task predictions.

Step-by-step Procedure

1. Choose a deep architecture.
2. Learn low-level features in early layers.
3. Compose them into higher-level features in later layers.
4. Train the network, often with backpropagation and regularization methods.

Formula and Calculation Details

No new general-purpose formula is introduced for deep learning itself in the lecture. The mathematical details come from the specific architectures studied next, especially autoencoders and convolutional neural networks.

Worked Mini Example

For handwritten digits:

1. Early layers may learn edges or strokes.
2. Middle layers may learn digit parts.
3. Later layers may learn entire digit shapes.
4. The final classifier predicts the digit class.

Strengths and Weaknesses

- Strengths: automatically learns layered features and performs very well on complex data.
- Weaknesses: data-hungry, computationally demanding, and often difficult to interpret.

Exam Talking Points

- "Deep learning means learning hierarchical feature representations with deep neural networks."
- "Its recent success is linked to larger datasets, stronger hardware, and improved architectures."

Common Mistakes

- Equating any neural network with deep learning.
- Forgetting that deep models may still need comparison against simple baselines.

1.24 Autoencoders

Core Idea

An autoencoder is a neural network trained to reconstruct its own input. A narrow hidden layer learns a compressed representation of the data.

Input and Output

- Input: unlabelled vectors x_i.
- Output: reconstructed vectors y_i and hidden representations h_i.

Step-by-step Procedure

1. Set each target equal to the input, so $y_i = x_i$.
2. Use fewer hidden units than input units for compression.
3. Train with backpropagation to reproduce the inputs.
4. Use the hidden-layer outputs as learned features or compressed codes.

Formula and Calculation Details

$$y_i = x_i$$

is the defining training target relation. The lecture also describes stacked autoencoders, where successive hidden representations are learned as:

$$h_1(x), \qquad h_2(h_1(x)), \ldots$$

Worked Mini Example

Suppose each image has 100 pixels and the hidden layer has 50 units.

1. Input dimension is 100.
2. The network is trained to output the same 100 values.
3. The hidden vector h has only 50 values.
4. Therefore h acts as a compressed representation of the image.

Strengths and Weaknesses

- Strengths: learns features from unlabelled data; useful for compression and pretraining.
- Weaknesses: learned features are not always better than hand-engineered ones; reconstruction quality may not equal task usefulness.

Exam Talking Points

- "An autoencoder is trained with target equal to input."
- "The hidden layer learns a compressed representation when it has fewer units than the input layer."

Common Mistakes

- Describing autoencoders as requiring class labels.
- Confusing reconstruction quality with classification accuracy.

1.25 Convolutional Neural Networks

Core Idea

CNNs use local connectivity, weight sharing, convolution, and pooling to learn visual features efficiently and with some translation invariance.

Input and Output

- Input: image-like arrays, often with multiple channels.
- Output: learned feature maps and final predictions.

Step-by-step Procedure

1. Slide a learned filter over local regions of the input.
2. Multiply corresponding entries and sum them to create a feature map.
3. Apply multiple filters to detect different features.
4. Use max-pooling to reduce size and gain some shift invariance.
5. Feed later features into fully connected layers for prediction.

Formula and Calculation Details

For a local filter response:

$$\text{feature value} = \sum_{i,j} x_{ij} w_{ij},$$

where x_{ij} are local input values and w_{ij} are filter weights. For max-pooling:

$$\text{pool output} = \max(x_1, \ldots, x_r).$$

The lecture describes local contrast normalization as subtracting a local mean and dividing by a local standard deviation:

$$x' = \frac{x - \mu}{\sigma}.$$

Worked Mini Example

With image patch

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

and filter

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix},$$

the convolution score is:

$$1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 = 5.$$

Strengths and Weaknesses

- Strengths: fewer parameters than fully connected image models; effective for spatial data; learns filters automatically.
- Weaknesses: deep models can be difficult to interpret and often require large data and computation.

Exam Talking Points

- "Weight sharing means the same filter is applied at many locations."
- "Pooling summarizes local responses and supports translation invariance."

Common Mistakes

- Thinking each image location has its own unrelated filter weights.
- Confusing convolutional filters with manually fixed features in trained CNNs.

1.26 Support Vector Machines

Core Idea

SVMs choose the separating hyperplane with the largest margin. Support vectors are the training examples that define that margin.

Input and Output

- Input: labelled training vectors, usually with labels $y_i \in \{-1, 1\}$.
- Output: a separating decision function for new examples.

Step-by-step Procedure

1. Represent the separating hyperplane.
2. Maximize the margin while correctly separating the data.
3. Solve the equivalent optimization problem.
4. Identify support vectors from non-zero Lagrange multipliers.
5. For non-linear data, replace dot products with a kernel function.

Formula and Calculation Details

Separating hyperplanes:

$$w \cdot x + b = 0, \qquad w \cdot x + b = 1, \qquad w \cdot x + b = -1.$$

Margin:

$$d = \frac{2}{\|w\|}.$$

Hard-margin primal optimization:

$$\min \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1.$$

Dual form shown in the lecture:

$$\max_{\lambda} \sum_i \lambda_i - \frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j (x_i \cdot x_j),$$

$$\lambda_i \geq 0, \qquad \sum_i \lambda_i y_i = 0,$$

$$w = \sum_i \lambda_i y_i x_i.$$

For the lecture's soft-margin extension, the same dual solution is used with an additional upper bound:

$$0 \leq \lambda_i \leq C,$$

where C controls the trade-off between margin size and training error. Classification:

$$f(z) = w \cdot z + b = \sum_i \lambda_i y_i (x_i \cdot z) + b, \qquad \hat{y} = \text{sign}(f(z)).$$

Kernel trick:

$$K(u, v) = \Phi(u) \cdot \Phi(v).$$

Examples of lecture kernels include:

$$K(x, y) = (x \cdot y + 1)^p, \qquad K(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right),$$

and a hyperbolic tangent kernel for some parameter choices.

Worked Mini Example

For two support vectors, suppose:

$$\lambda_1 = 0.5,\ y_1 = 1,\ x_1 = (1, 1), \qquad \lambda_2 = 0.5,\ y_2 = -1,\ x_2 = (-1, -1).$$

1. Compute:
$$w = 0.5(1)(1,1) + 0.5(-1)(-1,-1) = (1,1).$$

2. Let $b = 0$ and new point $z = (2,1)$.
$$f(z) = (1,1) \cdot (2,1) = 3.$$

3. Since $f(z) > 0$, predict class $+1$.

Strengths and Weaknesses

- Strengths: strong generalization, effective non-linear boundaries through kernels, only support vectors matter in the final classifier.
- Weaknesses: kernel and parameter selection matter; interpretation is less direct than with decision trees.

Exam Talking Points

- "SVMs maximize margin to reduce model complexity and improve generalization."
- "The kernel trick computes inner products in a transformed space without explicitly constructing that space."

Common Mistakes

- Saying all training examples define the final boundary equally.
- Confusing the maximum-margin hyperplane with any separating hyperplane.

1.27 Ensemble Learning

Core Idea

Ensemble methods combine several base classifiers. They work best when members are individually useful and make different mistakes.

Input and Output

- Input: training data and a way to generate multiple base classifiers.
- Output: a combined prediction, usually by voting or averaging.

Step-by-step Procedure

1. Generate diverse base learners.
2. Train each learner.
3. Collect predictions on a new example.
4. Combine predictions by majority vote, weighted vote, or averaging.

Formula and Calculation Details

For majority vote:

$$\hat{y} = \text{mode}(h_1(x), \ldots, h_M(x)).$$

For weighted voting:

$$\hat{y} = \arg\max_c \sum_{m=1}^{M} \alpha_m \mathbf{1}[h_m(x) = c].$$

Worked Mini Example

Three classifiers predict:

$$h_1(x) = \texttt{yes}, \quad h_2(x) = \texttt{yes}, \quad h_3(x) = \texttt{no}.$$

Majority vote predicts `yes`.

Strengths and Weaknesses

- Strengths: often improves predictive accuracy; can enlarge the effective hypothesis space.
- Weaknesses: harder to interpret; gains disappear when members are highly correlated or weak in the same way.

Exam Talking Points

- "Good ensembles need both accuracy and diversity among base classifiers."
- "Bagging, boosting, and random forests create diversity in different ways."

Common Mistakes

- Assuming many identical classifiers are better than one.
- Ignoring the role of error diversity.

1.28 Bagging

Core Idea

Bagging trains multiple models on bootstrap samples and combines them with equal voting.

Input and Output

- Input: dataset D with n examples and a base learner.
- Output: an ensemble prediction by majority vote or average.

Step-by-step Procedure

1. Sample n training examples with replacement to form one bootstrap dataset.
2. Train one classifier on that sample.
3. Repeat for M samples and M classifiers.
4. Predict a new example with every classifier.
5. Use majority vote for classification or average for regression.

Formula and Calculation Details

The lecture states that about 63% of original examples appear in a bootstrap sample. For regression combination:

$$\hat{y} = \frac{1}{M} \sum_{m=1}^{M} h_m(x).$$

Worked Mini Example

Original data IDs: $1, 2, 3, 4, 5$.

1. Bootstrap sample: $2, 5, 2, 1, 4$.
2. Train classifier h_1 on that sample.
3. Repeat to obtain h_2, h_3.
4. If predictions are `yes`, `no`, `yes`, majority vote is `yes`.

Strengths and Weaknesses

- Strengths: especially useful for unstable learners such as decision trees; easy to parallelize.
- Weaknesses: less helpful for already stable learners; less interpretable than a single model.

Exam Talking Points

- "Bagging means bootstrap aggregation."
- "It reduces variance by averaging over models trained on resampled data."

Common Mistakes

- Sampling without replacement.
- Using unequal voting weights when describing standard bagging.

1.29 Boosting

Core Idea

Boosting builds classifiers sequentially so that later learners focus more on examples that earlier learners found difficult.

Input and Output

- Input: labelled training data, weighted sampling scheme, and base learner.
- Output: a weighted ensemble classifier.

Step-by-step Procedure

1. Start with equal example weights.
2. Train the first classifier.
3. Increase weights of misclassified examples and decrease weights of correctly classified ones.
4. Train the next classifier on the reweighted data.
5. Continue for K rounds.
6. Combine classifiers by weighted vote.

Formula and Calculation Details

For the AdaBoost variant referenced in the lectures:

$$\epsilon_t = \sum_i p_i e_i,$$

where p_i is the current probability weight of example i and $e_i = 1$ if it is misclassified, else 0.

$$\beta_t = \frac{\epsilon_t}{1 - \epsilon_t}.$$

Correctly classified examples are down-weighted by β_t before normalization, and classifier vote weights can be written as:

$$\alpha_t = \log \frac{1}{\beta_t}.$$

Worked Mini Example

Ten examples start with weight 0.1 each. Suppose a classifier gets 3 wrong.

1. Weighted error:
$$\epsilon = 3(0.1) = 0.3.$$
2. Compute:
$$\beta = \frac{0.3}{0.7} \approx 0.429.$$
3. Correctly classified examples are multiplied by 0.429, so their weights fall before normalization.
4. Misclassified examples keep relatively larger weights, so the next learner focuses on them.

Strengths and Weaknesses

- Strengths: can turn weak learners into a strong ensemble; often very accurate.
- Weaknesses: sequential rather than embarrassingly parallel; sensitive to noisy labels and outliers.

Exam Talking Points

- "Boosting makes learners complementary by emphasizing previously misclassified examples."
- "Unlike bagging, boosting uses performance-dependent weights."

Common Mistakes

- Treating boosting as independent parallel training.
- Forgetting that the final vote is weighted.

1.30 Random Forest

Core Idea

Random forests combine bagging with random feature selection while growing many unpruned decision trees.

Input and Output

- Input: training data, number of trees M, and number of candidate features k per split.
- Output: majority-vote prediction across trees.

Step-by-step Procedure

1. Draw a bootstrap sample for each tree.
2. Grow each tree without pruning.
3. At every split, randomly choose only k features as candidates.
4. Split using the best feature among those k.
5. Predict by majority vote over all trees.

Formula and Calculation Details

The lecture gives the rule of thumb:

$$k = \log_2 m,$$

where m is the total number of available features.

Worked Mini Example

Suppose $m = 8$ features.

1. Choose $k = \log_2 8 = 3$ random features at each split.
2. Build many bootstrap-sampled trees.
3. If five trees vote `yes`, `no`, `yes`, `yes`, `no`, final prediction is `yes`.

Strengths and Weaknesses

- Strengths: often outperforms a single tree; random feature selection reduces correlation among trees.
- Weaknesses: lower interpretability; many trees increase storage and prediction cost.

Exam Talking Points

- "Random forest = bagging of random trees."
- "Random feature subsets reduce correlation while preserving tree strength."

Common Mistakes

- Pruning the trees when describing the standard random forest algorithm from the lecture.
- Using all features at every split.

1.31 Probabilistic Reasoning Basics

Core Idea

Probabilistic reasoning represents uncertainty numerically and answers queries by manipulating probability distributions.

Input and Output

- Input: random variables, known probabilities, and evidence.
- Output: requested marginal, joint, or conditional probabilities.

Step-by-step Procedure

1. Represent the relevant events.
2. Use the full joint distribution when available.
3. Sum out hidden variables when needed.
4. Apply conditional probability, product rule, chain rule, Bayes theorem, or total probability as appropriate.

Formula and Calculation Details

$$P(a \mid b) = \frac{P(a, b)}{P(b)}.$$

$$P(a, b) = P(a \mid b)P(b) = P(b \mid a)P(a).$$

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1}).$$

$$P(X) = \sum_i P(X \mid Y_i)P(Y_i).$$

$$P(B \mid A) = \frac{P(A \mid B)P(B)}{P(A)}$$

is Bayes theorem.

Worked Mini Example

If $P(M) = 1/50000$, $P(S \mid M) = 0.5$, and $P(S) = 1/20$, then:

$$P(M \mid S) = \frac{P(S \mid M)P(M)}{P(S)} = \frac{0.5(1/50000)}{1/20} = 0.0002.$$

Strengths and Weaknesses

- Strengths: principled handling of uncertainty.
- Weaknesses: full joint tables grow exponentially and are usually infeasible.

Exam Talking Points

- "Marginalization means summing over the unwanted variables."
- "Conditional independence is what makes large probabilistic models tractable."

Common Mistakes

- Reversing $P(A \mid B)$ and $P(B \mid A)$.
- Forgetting to normalize when deriving conditional probabilities.

1.32 Bayesian Networks

Core Idea

A Bayesian network is a directed acyclic graph that represents variables, dependencies, and conditional probability tables compactly.

Input and Output

- Input: variables, directed dependencies, and conditional probability tables.
- Output: a compact probabilistic model able to answer queries.

Step-by-step Procedure

1. Add one node per variable.
2. Add directed links according to direct influence.
3. Ensure the graph is acyclic.
4. Attach a conditional probability table to each node given its parents.
5. Obtain CPT values from experts or estimate them from data by counting frequencies.
6. Use the network factorization for probability calculations.

Formula and Calculation Details

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid \text{Parents}(x_i)).$$

The Bayesian-network independence assumption is:

$$P(\text{node} \mid \text{parents plus non-descendants}) = P(\text{node} \mid \text{parents}).$$

If each Boolean variable has at most k parents, a network needs:

$$O(n2^k)$$

numbers instead of $O(2^n)$ for a full joint table. For CPT learning by counting:

$$P(B) = \frac{\#b}{\#b + \#\neg b}, \qquad P(A \mid B, E) = \frac{\#a}{\#a + \#\neg a}$$

when the counts are restricted to rows where B and E take the stated values.

Worked Mini Example

For the burglary network:

$$P(j, m, a, \neg b, \neg e) = P(j \mid a)P(m \mid a)P(a \mid \neg b, \neg e)P(\neg b)P(\neg e).$$

With lecture values:

$$0.9 \cdot 0.7 \cdot 0.01 \cdot 0.999 \cdot 0.998 = 0.006281.$$

Strengths and Weaknesses

- Strengths: compact, interpretable, and explicitly represents conditional independence.
- Weaknesses: constructing a good structure and CPTs may require expertise or data; exact inference can still be expensive.

Exam Talking Points

- "A Bayesian network is a DAG plus CPTs."
- "Its main saving comes from conditional independence assumptions."
- "Naive Bayes is a special Bayesian-network topology with the class node as parent of all features."

Common Mistakes

- Adding cycles.
- Treating every absent edge as absolute independence without considering the stated conditional-independence semantics.

1.33 Exact Inference in Bayesian Networks

Core Idea

Exact inference computes posterior probabilities from a Bayesian network. Enumeration sums over hidden variables directly; variable elimination avoids repeated work by caching intermediate factors.

Input and Output

- Input: Bayesian network, query variables, and evidence variables.
- Output: a posterior distribution such as $P(X \mid E = e)$.

Step-by-step Procedure

1. Identify query, evidence, and hidden variables.
2. For enumeration, sum the joint probability over all hidden-variable assignments.
3. Normalize the resulting values.
4. For variable elimination, rewrite the same computation as factors.
5. Multiply and sum out variables one at a time, reusing intermediate factors.

Formula and Calculation Details

Enumeration:

$$P(X \mid E = e) = \alpha P(X, E = e) = \alpha \sum_{h} P(X, E = e, H = h),$$

where α normalizes the result. Variable elimination evaluates equivalent expressions factor by factor, for example:

$$P(B \mid j, m) = \alpha \sum_{e} \sum_{a} P(B)P(e)P(a \mid B, e)P(j \mid a)P(m \mid a).$$

Worked Mini Example

For the burglary example, the lecture computes:

$$P(b \mid j, m) = \alpha \cdot 0.00059224,$$
$$P(\neg b \mid j, m) = \alpha \cdot 0.0014919.$$

1. Compute normalization:

$$\alpha = \frac{1}{0.00059224 + 0.0014919}.$$

2. After normalization:

$$P(B \mid j, m) = (0.284, 0.716).$$

Strengths and Weaknesses

- Strengths: exact; variable elimination saves repeated computation compared with raw enumeration.
- Weaknesses: still expensive in large networks; elimination order affects intermediate factor size.

Exam Talking Points

- "Enumeration sums over every hidden-variable assignment."
- "Variable elimination performs the same mathematics more efficiently by reusing intermediate factors."

Common Mistakes

- Forgetting the normalization constant α.
- Eliminating query or evidence variables incorrectly.